



Duale Hochschule Baden-Württemberg Mannheim

Projektdokumentation

GuardX & SunShine Tours

Dokumentation der Web-Engineering-II-Projekte A und B

Studiengang Informatik - Cyber Security

Kurs MA-TINF25CS2

Team:

Gruppe2

Teammitglieder:

Moritz Brenner
Maik Baumgärtner
Marcel Reinholz
Jakob Goelden

Wiss. Betreuer:

Dipl.-Ing. Udo Erdmann

Abgabedatum:

30.06.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	iii
Quelltextverzeichnis	iii
Abkürzungsverzeichnis	iv
1 Einleitung	1
1.1 Zielsetzung der Dokumentation	1
1.2 Aufbau des Dokuments	1
I Projekt A: GuardX	2
2 Referenzdokument	2
2.1 Ziele	2
2.2 Funktionsrahmen	2
2.3 Projektrahmen	3
2.4 Umsetzungsstrategie	4
2.5 Aufgaben und Zuständigkeiten	4
3 Product-Backlog	5
4 Aufbau und Bedienbarkeit (Frontend)	7
4.1 Navigation	7
4.2 Startseite	7
4.3 Registrierung, Login und geschützte Bereiche	8
4.4 Fachmodule	8
4.5 Meldungen und Dialoge	9
5 Verzeichnisstruktur (Backend / Webserver)	10
6 Datenstrukturen und Tabellen (SQL-Server)	12
6.1 Tabelle user	12
6.2 Tabelle login_logs	12
6.3 Tabelle register_logs	13
7 Funktionale Umsetzung: Funktion und Skript	14
7.1 Sicherheitsmechanismen im Detail	16

8	Installation und Zugangsdaten	17
8.1	Systemvoraussetzungen	17
8.2	Installationsschritte	17
8.3	Zugangsdaten	17
II	Projekt B: SunShine Tours	18
9	Überblick und Umfang	18
9.1	Seitenübersicht	18
9.2	Gestaltung und technischer Umfang	19
10	Verwendete KI-Systeme und Werkzeuge	19
10.1	Weitere Werkzeuge	19
11	Funktionale Umsetzung: Funktion und Skript	20
11.1	Echtzeit-Kostenberechnung	21
11.2	Speicherung der Buchungen	21
12	Verzeichnisstruktur (Backend / Webserver)	22
13	Installation	24
13.1	Systemvoraussetzungen	24
13.2	Installationsschritte	24
13.3	Zugangsdaten	24
14	Fazit	25
	Literatur	26

Abbildungsverzeichnis

4.1	Startseite (Dashboard) mit Desktop-Navigation und den drei Modulkacheln . . .	7
4.2	Admin-Bereich mit Benutzerverwaltung	8

Tabellenverzeichnis

2.1	Technologischer Rahmen von Projekt A	3
2.2	Aufgaben und Zuständigkeiten in Projekt A	4
3.1	Product-Backlog Projekt A (GuardX)	5
5.1	Komponenten von Projekt A	11
6.1	Spalten der Tabelle user	12
6.2	Spalten der Tabelle login_logs	13
6.3	Spalten der Tabelle register_logs	13
7.1	Zuordnung Funktionalität zu Skript (Projekt A)	14
8.1	Zugangsdaten Projekt A	17
10.1	Eingesetzte KI-Systeme in Projekt B	19
11.1	Zuordnung Funktionalität zu Skript (Projekt B)	20
12.1	Komponenten von Projekt B	22

Quelltextverzeichnis

6.1	CREATE-Statement der Tabelle user	12
7.1	Zeitkonstante CSRF-Prüfung (Auszug admin.php)	16
11.1	Kern der Kostenberechnung (Auszug js/booking.js)	21

Abkürzungsverzeichnis

AGB	Allgemeine Geschäftsbedingungen
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
CIDR	Classless Inter-Domain Routing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DOM	Document Object Model
EXIF	Exchangeable Image File Format
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PHP	PHP: Hypertext Preprocessor
SQL	Structured Query Language
XAMPP	Cross-Platform, Apache, MariaDB, PHP, Perl
XSS	Cross-Site Scripting

1 Einleitung

Diese Dokumentation fasst die beiden im Rahmen der Vorlesung *Web-/App-Engineering II* (Kurs MA-TINF25CS2, Sommersemester 2026) von **Gruppe2** entwickelten Projekte zusammen. Beide Projekte wurden als Teamarbeit umgesetzt; die Dokumentation ist bewusst als *ein* gemeinsames Dokument angelegt und in zwei Teile gegliedert.

Teil I beschreibt das Projekt A *GuardX* – eine datenbankgestützte Web-App mit Session- und Benutzerverwaltung, Admin-Bereich und mehreren sicherheitsorientierten Werkzeugen. **Teil II** beschreibt das Projekt B *SunShine Tours* – eine responsive Web-Seite für ein (fiktives) Städte-Tour-Unternehmen mit Echtzeit-Kostenberechnung und dateibasierter Buchungsverwaltung. Während Projekt A den Schwerpunkt auf Sicherheit, Sessions und Datenbankzugriffe legt, steht bei Projekt B die clientseitige Interaktivität sowie die KI-gestützte Erstellung sämtlicher Inhalte und Grafiken im Vordergrund.

1.1 Zielsetzung der Dokumentation

Ziel dieser Dokumentation ist es, beide Anwendungen so zu beschreiben, dass sie auf einem fremden System nachvollziehbar installiert, gestartet und bewertet werden können. Dafür werden je Projekt die Funktionalität, der Aufbau des Frontends, die Verzeichnis- und ggf. Datenstruktur sowie alle benötigten Zugangsdaten dargestellt. Besonderer Wert wird auf die Nachvollziehbarkeit gelegt: Für jede geforderte Funktionalität wird in einer Zuordnungstabelle angegeben, in welchem Skript sie umgesetzt ist (vgl. Kapitel 7 bzw. 11).

1.2 Aufbau des Dokuments

Nach dieser Einleitung gliedert sich das Dokument wie folgt. Teil I (Projekt A) beginnt mit einem Referenzdokument (Ziele, Funktionsrahmen, Projektrahmen, Umsetzungsstrategie sowie Aufgaben und Zuständigkeiten), gefolgt vom Product-Backlog, der Beschreibung von Frontend, Verzeichnisstruktur und Datenbank, der funktionalen Umsetzung sowie einer Installationsanleitung inklusive Zugangsdaten. Teil II (Projekt B) beschreibt analog Überblick und Umfang, die eingesetzten KI-Systeme und Werkzeuge, die funktionale Umsetzung, die Verzeichnisstruktur und die Installation. Den Abschluss bildet ein kurzes Fazit.

Teil I

Projekt A: GuardX

2 Referenzdokument

2.1 Ziele

GuardX ist ein modulares „Web Security & Privacy Toolkit“. Ziel des Projekts ist eine Web-App, die registrierten Nutzerinnen und Nutzern mehrere praxisnahe Werkzeuge rund um digitale Sicherheit und Privatsphäre an einer Stelle bereitstellt und dabei selbst nach gängigen Sicherheitsprinzipien aufgebaut ist. Die App verfolgt damit einen doppelten Zweck: Zum einen liefert sie konkreten Nutzen (Passwörter prüfen, Bild-Metadaten entfernen, Tracking-Risiken verstehen), zum anderen demonstriert sie die im Modul geforderten technischen Konzepte – insbesondere Sessionverwaltung, sichere Authentifizierung, Rechteverwaltung und Datenbankzugriffe.

2.2 Funktionsrahmen

Die Anwendung gliedert sich in einen **Account- und Verwaltungsbereich** und drei **Fachmodule**.

Der Account- und Verwaltungsbereich umfasst:

- Registrierung, Login und Logout mit serverseitiger Validierung,
- Sessionverwaltung mit automatischem Logout nach konfigurierbarer Inaktivität,
- einen Admin-Bereich mit vollständiger Benutzerverwaltung (Create, Read, Update, Delete (CRUD)),
- Passwortverwaltung (Ändern durch Nutzer, Zurücksetzen durch Admin) und

- eine Rechteverwaltung mit den Rollen *Admin* und *User*.

Die drei Fachmodule sind:

- **Metadaten-Bereiniger:** Entfernt Exchangeable Image File Format (EXIF)-Daten aus hochgeladenen Bildern und bietet die ausgelesenen Metadaten als JavaScript Object Notation (JSON)-Datei zum Download an.
- **Browser-Fingerprinting:** Macht sichtbar, welche Merkmale (Canvas, WebGL, Zeitzone, Sprache, Hardware) einen Browser identifizierbar machen.
- **Live-Passwort-Checker:** Prüft per Asynchronous JavaScript and XML (AJAX) in Echtzeit die Passwortstärke und gleicht das Passwort datenschutzfreundlich (k-Anonymity) gegen bekannte Datenlecks ab.

Querschnittlich werden die geforderten Technologien eingesetzt: client- und serverseitige Eingabeprüfung (u. a. mit regulären Ausdrücken), Einbindung von jQuery/jQuery UI und Bootstrap, dynamisches Nachladen via AJAX, Meldungs- und Bestätigungsdialoge, Datenexport via JSON sowie eine dynamische, responsive Menüstruktur. Eine vollständige Zuordnung dieser Funktionen zu den jeweiligen Skripten findet sich in Kapitel 7.

2.3 Projektrahmen

Das Projekt wurde als zeitlich begrenzte Teamarbeit von vier Personen im Sommersemester 2026 umgesetzt. Die technologischen Rahmenbedingungen entsprechen den Projektvorgaben:

Tabelle 2.1: Technologischer Rahmen von Projekt A

Bereich	Technologie
Client	HTML5, CSS, JavaScript; jQuery, jQuery UI, Bootstrap 5.3
Server	Apache, PHP 8.2.4 (ohne Framework)
Datenbank	MariaDB 10.4.28
Laufzeitumgebung	XAMPP
Versionsverwaltung	Git / GitHub

Auf den Einsatz fertiger Frameworks (z. B. Angular, Vue, Symfony, React) wurde vorgabengemäß verzichtet; eingesetzt werden ausschließlich die erlaubten Bibliotheken jQuery, jQuery UI und Bootstrap.

2.4 Umsetzungsstrategie

Die Entwicklung erfolgte iterativ und arbeitsteilig. Zur Zusammenarbeit wurde ein Git-Feature-Branch-Workflow genutzt: Neue Funktionen wurden in eigenen Branches entwickelt, per Pull-Request zusammengeführt und vor dem Merge daraufhin geprüft, ob die Anwendung unter XAMPP lauffähig bleibt. Wiederkehrende Bestandteile (Header/Navigation, Footer, Konfiguration) wurden als wiederverwendbare Includes ausgelagert, sodass alle Seiten ein einheitliches Erscheinungsbild und Verhalten besitzen. Sicherheitsrelevante Mechanismen (Cross-Site Request Forgery (CSRF)-Schutz, Passwort-Hashing, Prepared Statements) wurden von Beginn an mitgedacht und sind zentral umgesetzt. Vor der Abgabe wurde die Installierbarkeit auf einem frischen System getestet.

2.5 Aufgaben und Zuständigkeiten

Die Arbeitspakete wurden wie in Tabelle 2.2 dargestellt verteilt. Die Zuordnung orientiert sich an den jeweils verantworteten Komponenten; übergreifende Aufgaben (Code-Reviews, Integration, Test) wurden gemeinsam wahrgenommen.

Tabelle 2.2: Aufgaben und Zuständigkeiten in Projekt A

Teammitglied	Verantwortungsbereich
Maik Baumgärtner	Backend: Session- und Benutzerverwaltung, Login/Logout/Registrierung, Admin-Bereich, Datenbankbindung und -schema, zentrale Hilfsfunktionen sowie Navigations-Templates.
Marcel Reinholz	Footer und alle darüber verlinkten Rechtsseiten (Impressum, Datenschutz, AGB) sowie das Modul der API-Aufrufe (Live-Passwort-Checker, Leak-Abgleich).
Jakob Goelden	Modul Browser-Fingerprinting (<code>info.php</code>) sowie Einrichtung und Pflege des GitHub-Repositories (Branch-/Merge-Strategie, Versionsverwaltung).
Moritz Brenner	Modul Metadaten-Bereiniger, das durchgängige CSS-Design (Layout, Cyber-Theme, responsive Anpassungen) sowie die Startseite.

3 Product-Backlog

Das Product-Backlog wurde über die gesamte Projektlaufzeit gepflegt und priorisiert. Es bildet die umgesetzten Anforderungen ab und ordnet jeder Anforderung eine Priorität, eine Aufwandschätzung in Story Points (SP), die verantwortliche Person sowie den Bearbeitungsstatus zu. Zum Abgabzeitpunkt sind alle Einträge umgesetzt. Tabelle 3.1 zeigt das vollständige Backlog.

Tabelle 3.1: Product-Backlog Projekt A (GuardX)

Nr.	Anforderung	Prio	SP	Zuständig	Status
A-01	Sessionverwaltung inkl. CSRF-Token-Erzeugung	Hoch	5	Maik	Erledigt
A-02	Registrierung mit serverseitiger Prüfung und Passwort-Hashing	Hoch	5	Maik	Erledigt
A-03	Login/Logout mit Rollenweiterleitung (Admin/User)	Hoch	5	Maik	Erledigt
A-04	Automatisches Logout nach konfigurierbarer Inaktivität	Hoch	3	Maik	Erledigt
A-05	CSRF-Schutz für alle schreibenden Formulare	Hoch	3	Maik	Erledigt
A-06	Brute-Force-Schutz: Login-Sperre nach Fehlversuchen	Mittel	3	Maik	Erledigt
A-07	Rate-Limiting der Registrierung pro IP-Adresse	Mittel	3	Marcel	Erledigt
A-08	Admin: Benutzer anlegen und löschen	Hoch	5	Maik	Erledigt
A-09	Admin: Rolle umschalten, E-Mail und Passwort setzen	Hoch	5	Maik	Erledigt
A-10	Rechteverwaltung: Zugriffsschutz für Admin-Seiten	Hoch	3	Maik	Erledigt
A-11	Self-Service: E-Mail/Passwort ändern, Account löschen	Mittel	5	Maik	Erledigt
A-12	Zentrale Konfiguration über Konfigurationsdatei	Mittel	2	Maik	Erledigt

Fortsetzung nächste Seite

Tabelle 3.1 – Fortsetzung

Nr.	Anforderung	Prio	SP	Zuständig	Status
A-13	Dynamische, responsive Navigation (Desktop/Mobile)	Hoch	5	Maik	Erledigt
A-14	Session-Timeout-Warndialog mit jQuery UI	Mittel	3	Maik	Erledigt
A-15	Modul Metadaten-Bereiniger: EXIF-Daten entfernen	Hoch	8	Moritz	Erledigt
A-16	Ausgelesene EXIF-Daten als JSON-Export anbieten	Mittel	3	Moritz	Erledigt
A-17	Automatische Bildrotation anhand EXIF-Orientierung	Niedrig	3	Moritz	Erledigt
A-18	Modul Browser-Fingerprinting (Canvas, WebGL, u. a.)	Hoch	8	Jakob	Erledigt
A-19	Modul Live-Passwort-Checker (AJAX, Regex-Stärke)	Hoch	8	Marcel	Erledigt
A-20	Leak-Abgleich via HavelBeenPwned (k-Anonymity)	Mittel	5	Marcel	Erledigt
A-21	Rechtsseiten Impressum/Datenschutz/AGB inkl. Footer	Mittel	3	Marcel	Erledigt
A-22	Durchgängiges CSS-Design (Cyber-Theme, responsive)	Mittel	8	Moritz	Erledigt
A-23	Sichere IP-Ermittlung hinter Proxy/Cloudflare	Niedrig	3	Maik	Erledigt
A-24	Git-Repository, Branch- und Merge-Workflow	Mittel	3	Jakob	Erledigt
A-25	Abgabevorbereitung (SQL-Dump, php.ini-Hinweise, Test)	Hoch	3	Alle	Erledigt

4 Aufbau und Bedienbarkeit (Frontend)

Das Frontend von GuardX folgt durchgängig der klassischen Struktur aus Kopfbereich mit Navigation, Inhaltsbereich und Footer. Gestalterisch kommt ein dunkles „Cyber-Theme“ mit grünen Akzenten zum Einsatz, das auf allen Seiten über eine gemeinsame Stylesheet-Basis (`style/main.css`, `style/navbar.css`) erzeugt wird.

4.1 Navigation

Die Navigation ist sowohl *dynamisch* als auch *responsiv* umgesetzt. Serverseitig wird in jedem Seitenaufruf über die Funktion `is_mobile()` anhand des User-Agents entschieden, ob die Desktop-Navigation (`template/navbar.php`) oder die mobile Variante mit Hamburger-Menü (`template/navbar_mobile.php`) eingebunden wird. Die angezeigten Menüpunkte hängen zusätzlich vom Login-Status und der Rolle ab: Nicht angemeldete Nutzer sehen einen *Login*-Punkt, angemeldete Nutzer den Bereich *Mein Bereich* und Administratoren zusätzlich *System Admin*. Die Links werden über die Hilfsfunktion `get_url()` erzeugt, damit sie unabhängig vom Installationsverzeichnis korrekt auflösen.



Abbildung 4.1: Startseite (Dashboard) mit Desktop-Navigation und den drei Modulkacheln

4.2 Startseite

Die Startseite (`index.php`) dient als Dashboard und stellt die drei Fachmodule als Kacheln mit kurzer Beschreibung vor: Metadaten-Bereiniger, Browser-Fingerprinting und Live-Passwort-Checker. Von hier aus gelangt man per Klick direkt in das jeweilige Modul.

4.3 Registrierung, Login und geschützte Bereiche

Über `auth.php` erfolgen Registrierung und Login in einem gemeinsamen, umschaltbaren Formular. Nach erfolgreichem Login wird der Nutzer rollenabhängig weitergeleitet – als Admin in das Admin-Dashboard, sonst in den persönlichen Bereich. Rückmeldungen (Erfolg, Fehler, Sperrhinweise) werden direkt am Formular angezeigt.

Der **persönliche Bereich** (`user.php`) erlaubt das Ändern der E-Mail-Adresse und des Passworts (mit Bestätigung des aktuellen Passworts) sowie das Löschen des eigenen Accounts. Sicherheitskritische Aktionen werden über einen Bestätigungsdialog abgesichert.

Der **Admin-Bereich** (`admin.php`) bietet eine tabellarische Benutzerübersicht mit Inline-Formularen, über die Administratoren neue Benutzer anlegen, E-Mail-Adressen und Passwörter setzen, die Rolle umschalten und Benutzer löschen können. Der eigene Account ist gegen versehentliches Löschen geschützt.

The screenshot displays the 'SYSTEM_ADMIN' interface. At the top, there's a section for '[+] Neuen User registrieren' (New User Registration) with input fields for Username (admin_02), E-Mail (user@domain.com), Password (masked with asterisks), and a Role dropdown menu (set to User). A green 'Erstellen' (Create) button is below these fields.

Below this is the '[=] User Datenbank' (User Database) section, which contains a table of users:

ID	USERNAME	E-MAIL	NEUES PASSWORT	ROLLE	AKTIONEN
#12	UdoAdmin		Save	Neues PW... Set	ADMIN [Dein Account]
#11	UdoErdmann		Save	Neues PW... Set	USER -> Admin Löschen
#10	test12		Save	Neues PW... Set	USER -> Admin Löschen

Abbildung 4.2: Admin-Bereich mit Benutzerverwaltung

4.4 Fachmodule

Metadaten-Bereiniger. Über ein Upload-Formular wird ein Bild hochgeladen; die Anwendung liest die EXIF-Daten aus, bietet sie als JSON-Datei zum Download an und liefert eine von Metadaten bereinigte Kopie des Bildes zurück. Das Original wird nach der Verarbeitung serverseitig gelöscht.

Browser-Fingerprinting. Die Seite (`info.php`) stellt anschaulich dar, welche Merkmale einen Browser identifizierbar machen (u. a. Canvas- und WebGL-Fingerprint, Zeitzone, Sprache, Hardware). Die Werte werden clientseitig per JavaScript ermittelt und hervorgehoben.

Live-Passwort-Checker. Bei jeder Eingabe wird das Passwort per AJAX an den Server geschickt und sofort bewertet: Die Anforderungen an die Stärke (Länge, Groß-/Kleinbuchstaben, Ziffern, Sonderzeichen) werden mit regulären Ausdrücken geprüft, zusätzlich erfolgt ein datenschutzfreundlicher Abgleich gegen bekannte Datenlecks. Ein Auge-Symbol blendet das Passwort bei Bedarf ein.

4.5 Meldungen und Dialoge

Für Hinweise und Bestätigungen werden zwei Mechanismen verwendet: einfache `confirm()`-Dialoge für destruktive Aktionen (z. B. Benutzer löschen) sowie ein modaler jQuery-UI-Dialog, der vor Ablauf der Session erscheint und die Wahl zwischen *Bleiben* und *Ausloggen* bietet.

5 Verzeichnisstruktur (Backend / Webserver)

Das Projektverzeichnis wird vollständig in das Webserver-Wurzelverzeichnis (bei XAMPP htdocs) installiert. Alle internen Pfade sind relativ aufgelöst, sodass das Projekt unabhängig vom Ordernamen lauffähig ist. Die folgende Übersicht zeigt die Struktur:

```
guardx/
|-- index.php Startseite / Dashboard
|-- init.php Session-Start, Timeout, CSRF-Token
|-- config.php Zentrale Konfiguration (DB-Verbindung)
|-- functions.php Server-Hilfsfunktionen (IP, Pfade, Mobile)
|-- functions.js Client-Logik (Session-Timer, Menue)
|-- auth.php Login / Registrierung / Logout
|-- user.php Persoenlicher Bereich (Profil)
|-- admin.php Admin-Bereich (Benutzerverwaltung, CRUD)
|-- admin_logout.php Logout-Endpunkt
|-- keep_alive.php Session am Leben halten (AJAX-Ziel)
|-- impressum.php Impressum
|-- datenschutz.php Datenschutzerklaerung
|-- agb.php AGB
|-- php.ini.hinweise Hinweise zu benoetigten PHP-Extensions
|
|-- db/
| |-- users.sql Aktueller DB-Dump (Struktur + Daten)
| |-- init_databases.sql Aelterer Dump (siehe Hinweis Kap. 8)
|
|-- include/
| |-- api-calls/
| | |-- skript.php Oberflaeche Live-Passwort-Checker
| | |-- check.php AJAX-Endpunkt: Staerke + Leak-Abgleich
| |-- fingerprinting/
| | |-- info.php Modul Browser-Fingerprinting
| |-- metadata_stripping/
| |-- metadata_stripping.php Modul Metadaten-Bereiniger
| |-- uploads/ (zur Laufzeit erzeugt) Upload-Ablage
| |-- clean/ (zur Laufzeit erzeugt) bereinigte Bilder + JSON
|
|-- template/
| |-- navbar.php Desktop-Navigation
| |-- navbar_mobile.php Mobile Navigation (Hamburger)
| |-- footer.php Footer mit Rechts-Links
|
|-- style/
```

```

|-- main.css Basis-/Layout-Styles (Cyber-Theme)
|-- navbar.css Navigationsstyles
|-- metadata_stripping.css Styles des Metadaten-Moduls

```

Die Verzeichnisse `include/metadata_stripping/uploads/` und `.../clean/` werden vom Modul bei Bedarf automatisch angelegt; sie müssen für den Webserver-Benutzer beschreibbar sein. Tabelle 5.1 fasst die Aufgaben der wichtigsten Komponenten zusammen.

Tabelle 5.1: Komponenten von Projekt A

Datei / Verzeichnis	Aufgabe
<code>init.php</code>	Startet/erneuert die Session, erzwingt das Inaktivitäts-Timeout und erzeugt das CSRF-Token. Wird von praktisch allen Seiten eingebunden.
<code>config.php</code>	Zentrale Konfigurationsdatei mit den Verbindungsdaten zur Datenbank; wird per <code>require_once</code> eingebunden.
<code>functions.php</code>	Serverseitige Hilfsfunktionen: sichere IP-Ermittlung, Pfad-Helfer (<code>get_url</code>), Mobile-Erkennung (<code>is_mobile</code>).
<code>functions.js</code>	Clientseitige Logik: Session-Timer mit jQuery-UI-Dialog, mobiles Menü, Login-Weiterleitung.
<code>auth.php</code>	Registrierung, Login und Logout inkl. Validierung, Hashing, Brute-Force- und Rate-Limit-Schutz.
<code>user.php</code>	Persönlicher Bereich: E-Mail/Passwort ändern, Account löschen.
<code>admin.php</code>	Admin-Bereich mit vollständiger Benutzerverwaltung (Anlegen, Ändern, Rolle, Löschen).
<code>include/api-calls/</code>	Live-Passwort-Checker: Oberfläche (<code>skript.php</code>) und AJAX-Endpunkt (<code>check.php</code>).
<code>include/fingerprinting/</code>	Modul Browser-Fingerprinting (<code>info.php</code>).
<code>metadata_stripping/</code>	Modul Metadaten-Bereiniger inkl. Laufzeit-Ordern für Upload und bereinigte Ausgabe.
<code>template/</code>	Wiederverwendbare Bausteine: Desktop-/Mobil-Navigation und Footer.
<code>style/</code>	Stylesheets für Layout, Navigation und Metadaten-Modul.
<code>db/</code>	SQL-Dumps der Datenbank (Struktur und Daten).

6 Datenstrukturen und Tabellen (SQL-Server)

Die Anwendung nutzt die MariaDB-Datenbank `users`. Sie umfasst drei Tabellen: `user` für die Benutzerkonten sowie `login_logs` und `register_logs` für sicherheitsrelevante Protokollierung. Alle Tabellen besitzen einen aufsteigenden Primärschlüssel `id`. Zwischen `login_logs.user_id` und `user.id` besteht eine logische Zuordnung (Fremdschlüsselbeziehung), die in der Anwendung ausgewertet wird.

6.1 Tabelle `user`

Speichert die Benutzerkonten. Das Passwort wird ausschließlich als bcrypt-Hash abgelegt; `is_admin` steuert die Rolle.

Tabelle 6.1: Spalten der Tabelle `user`

Spalte	Typ	Bedeutung
<code>id</code>	<code>INT, PK, AUTO_INCREMENT</code>	Eindeutige Benutzer-ID
<code>username</code>	<code>TEXT, NOT NULL</code>	Anmeldename (eindeutig geprüft)
<code>email</code>	<code>VARCHAR(100), NULL</code>	Optionale E-Mail-Adresse
<code>password</code>	<code>VARCHAR(255), NOT NULL</code>	bcrypt-Hash des Passworts
<code>is_admin</code>	<code>TINYINT(1), Default 0</code>	Rolle: 1 = Admin, 0 = User
<code>zeit</code>	<code>DATE, Default heute</code>	Anlagedatum des Kontos

```
1 CREATE TABLE `user` (  
2   `id` int(11) NOT NULL,  
3   `username` text NOT NULL,  
4   `email` varchar(100) DEFAULT NULL,  
5   `password` varchar(255) NOT NULL,  
6   `is_admin` tinyint(1) NOT NULL DEFAULT 0,  
7   `zeit` date NOT NULL DEFAULT current_timestamp()  
8 ) ENGINE=InnoDB DEFAULT CHARSET=latin1 COLLATE=latin1_swedish_ci;
```

Quelltext 6.1: CREATE-Statement der Tabelle `user`

6.2 Tabelle `login_logs`

Protokolliert jeden Login-Versuch. Anhand der fehlgeschlagenen Versuche eines Nutzers innerhalb eines Zeitfensters wird die Brute-Force-Sperre umgesetzt.

Tabelle 6.2: Spalten der Tabelle login_logs

Spalte	Typ	Bedeutung
id	INT, PK, AUTO_INCREMENT	Eindeutige Log-ID
user_id	INT, NOT NULL	Bezug auf user.id
ip_address	VARCHAR(45)	IP-Adresse (IPv4/IPv6)
login_time	DATETIME, Default jetzt	Zeitpunkt des Versuchs
success	TINYINT(1), Default 1	1 = erfolgreich, 0 = fehlgeschlagen

6.3 Tabelle register_logs

Protokolliert Registrierungen je IP-Adresse und ermöglicht so das Rate-Limiting (maximale Anzahl neuer Konten pro IP und Tag).

Tabelle 6.3: Spalten der Tabelle register_logs

Spalte	Typ	Bedeutung
id	INT, PK, AUTO_INCREMENT	Eindeutige Log-ID
ip_address	VARCHAR(45)	IP-Adresse der Registrierung
attempt_time	DATETIME, Default jetzt	Zeitpunkt der Registrierung

Alle Datenbankzugriffe erfolgen ausschließlich über *Prepared Statements*, um SQL-Injection zu verhindern [4]. Die vollständige Struktur inklusive Beispieldaten liegt im Dump db/users.sql (vgl. Kapitel 8).

7 Funktionale Umsetzung: Funktion und Skript

Dieses Kapitel ordnet jede geforderte Funktionalität dem Skript bzw. der Datei zu, in der sie umgesetzt ist. Damit ist nachvollziehbar, wo eine bestimmte Funktion zu finden ist (vgl. Abgabevorgabe zur Nachvollziehbarkeit). Tabelle 7.1 gibt die vollständige Zuordnung an.

Tabelle 7.1: Zuordnung Funktionalität zu Skript (Projekt A)

Funktionalität	Skript / Datei	Kurzbeschreibung
Account, Session und Rechteverwaltung		
Sessionverwaltung	init.php	Session-Start und -Wiederherstellung, CSRF-Token
Login / Logout	auth.php, admin_logout.php	Anmeldung, Rollenweiterleitung, Abmeldung
Registrierung	auth.php	Serverseitige Prüfung neuer Konten
Automatisches Logout	init.php, functions.js	Timeout-Prüfung (Server) und Warndialog (Client)
Benutzerverwaltung (CRUD)	admin.php	Anlegen, Ändern, Rolle, Löschen
Rechteverwaltung (Rollen)	init.php, admin.php	Zugriffsschutz über is_admin
Self-Service-Profil	user.php	E-Mail/Passwort ändern, Account löschen
Zentrale Konfiguration	config.php	DB-Zugangsdaten, per require_once
Sicherheit		
CSRF-Schutz	init.php (+ Prüfung in admin.php, user.php, auth.php)	Token-Erzeugung und -Prüfung je Formular

Fortsetzung nächste Seite

Tabelle 7.1 – Fortsetzung

Funktionalität	Skript / Datei	Kurzbeschreibung
Passwort-Hashing	auth.php	bcrypt via password_hash/verify
Brute-Force-Schutz	auth.php, Tabelle login_logs	Sperre nach mehreren Fehlversuchen
Rate-Limiting (Registrierung)	auth.php, Tabelle register_logs	Begrenzung neuer Konten pro IP
Prepared Statements	auth.php, admin.php, user.php	Schutz vor SQL-Injection
Eingabevalidierung (RegExp)	functions.js, check.php, auth.php	Client- und serverseitig, reguläre Ausdrücke
Oberfläche, Bibliotheken und AJAX		
jQuery / jQuery UI	functions.js	Timeout-Dialog, DOM-Interaktion
Bootstrap-Einbindung	template/, style/	Layout- und Komponentenraster
AJAX-Nachladen	api-calls/check.php, keep_alive.php	Live-Prüfung, Session-Verlängerung
Meldungs- /Bestätigungsdialoge	functions.js, template/footer.php	confirm() und jQuery-UI-Dialog
Dynamische, responsive Navigation	template/navbar.php, navbar_mobile.php	rollenabhängig, is_mobile(), get_url()
Fachmodule und weitere Seiten		
JSON-Datenexport	metadata_stripping.php	EXIF-Daten als JSON-Download
Datei-Upload	metadata_stripping.php	Bild-Upload und -Bereinigung
Externer API-Aufruf	api-calls/check.php	Leak-Abgleich (HaveIBeenPwned)
Browser-Fingerprinting	fingerprinting/info.php	Canvas, WebGL, Zeitzone u. a.

Fortsetzung nächste Seite

Tabelle 7.1 – Fortsetzung

Funktionalität	Skript / Datei	Kurzbeschreibung
Rechtsseiten	impressum.php, datenschutz.php, agb.php	über den Footer verlinkt

7.1 Sicherheitsmechanismen im Detail

Die sicherheitsrelevanten Funktionen sind bewusst zentral umgesetzt:

CSRF-Schutz. Für jede Session wird in `init.php` ein Zufalls-Token erzeugt. Jedes schreibende Formular führt das Token mit; serverseitig wird es zeitkonstant geprüft, bevor eine Aktion ausgeführt wird [2].

```

1 if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
2     die("CSRF Token ungültig!");
3 }

```

Quelltext 7.1: Zeitkonstante CSRF-Prüfung (Auszug `admin.php`)

Password-Hashing. Passwörter werden nie im Klartext gespeichert, sondern mit dem bcrypt-Verfahren über `password_hash()` gehasht und beim Login mit `password_verify()` geprüft [3].

Prepared Statements. Sämtliche Datenbankabfragen verwenden parametrisierte Abfragen, so dass Benutzereingaben nicht als SQL interpretiert werden können.

Brute-Force- und Rate-Limiting. Fehlgeschlagene Logins werden in `login_logs` protokolliert und führen nach mehreren Versuchen zu einer temporären Sperre; Registrierungen werden über `register_logs` je IP begrenzt.

Datenschutzfreundlicher Leak-Abgleich. Beim Passwort-Check wird nur ein Präfix des SHA-1-Hashes an die externe API gesendet (k-Anonymity); der Abgleich der vollständigen Hashes erfolgt lokal [1].

8 Installation und Zugangsdaten

8.1 Systemvoraussetzungen

GuardX setzt eine Standard-XAMPP-Umgebung voraus: Apache als Webserver, PHP 8.2 sowie MariaDB. Zusätzlich müssen einige PHP-Erweiterungen aktiv sein (siehe unten).

8.2 Installationsschritte

1. **Projekt kopieren.** Den gesamten Projektordner in das Webserver-Wurzelverzeichnis legen (bei XAMPP: C:\xampp\htdocs).
2. **PHP-Erweiterungen aktivieren.** In der `php.ini` (bei XAMPP C:\xampp\php\php.ini) müssen die Erweiterungen `gd`, `mbstring`, `exif` und `curl` aktiv (nicht auskommentiert) sein. Anschließend Apache neu starten. Die benötigten Einträge sind in der mitgelieferten Datei `php.ini.hinweise` dokumentiert.
3. **Datenbank importieren.** In phpMyAdmin eine Datenbank `users` anlegen und den Dump `db/users.sql` importieren. Damit werden Struktur *und* Beispieldaten angelegt.
4. **Schreibrechte prüfen.** Die Verzeichnisse unter `include/metadata_stripping/` (`uploads/`, `clean/`) müssen für den Webserver beschreibbar sein. Sie werden bei Bedarf automatisch angelegt.
5. **Aufruf.** Die Anwendung über `http://localhost/<Projektordner>/index.php` im Browser öffnen.

8.3 Zugangsdaten

Tabelle 8.1 listet alle für Betrieb und Bewertung notwendigen Zugangsdaten.

Tabelle 8.1: Zugangsdaten Projekt A

Zweck	Benutzer	Passwort / Wert
Datenbank (MariaDB)	root	(leer – XAMPP-Standard)
Datenbankname	users (Host: localhost)	
Demo-Login (User)	UdoErdmann	AbgabeGruppe2
Demo-Login (Admin)	UdoAdmin	AbgabeGruppe2

Teil II

Projekt B: SunShine Tours

9 Überblick und Umfang

SunShine Tours ist eine als Demonstrationsprojekt umgesetzte Web-Seite für einen fiktiven Anbieter von Städtereisen. Die inhaltliche Ausgestaltung ist bewusst satirisch gehalten; technisch handelt es sich um eine serverseitig mit PHP erzeugte Web-Anwendung, die ohne Datenbank auskommt und ihre Daten dateibasiert (im JSON-Format) ablegt.

9.1 Seitenübersicht

Die Anwendung besteht aus mehreren Inhaltsseiten mit gemeinsamem Kopf- und Fußbereich:

- **Startseite** (`index.php`) – Einstieg mit Hero-Bereich und Kennzahlen.
- **Über uns** (`about.php`) – Vorstellung des (fiktiven) Teams und der Unternehmenswerte.
- **Touren / Kundenstimmen** (`tours.php`) – Tourangebote und mehrere Kundenbewertungen.
- **Buchung** (`booking.php`) – Buchungsformular mit Echtzeit-Kostenanzeige.
- **Bestätigung** (`confirmation.php`) – Zusammenfassung nach erfolgreicher Buchung.
- **Rechtliches** (`legal.php`) – Impressum, Datenschutz und AGB.
- **Formular-Test** (`formcheck.php`) – kleine Seite zur Demonstration der GET-/POST-Verarbeitung.

9.2 Gestaltung und technischer Umfang

Das Layout folgt der Struktur Kopfbereich–Inhalt–Fußbereich und ist responsiv umgesetzt. Die Farbgebung (Gelb-, Orange- und Grüntöne) ist über CSS-Variablen zentral definiert, wodurch sich auch ein Hell-/Dunkel-Modus realisieren lässt. Das Logo ist vollständig in CSS gestaltet (ohne Bilddatei). Die auf den Seiten verwendeten Städtebilder wurden mit einem KI-Bildgenerator erzeugt (vgl. Kapitel 10).

Der technische Umfang umfasst sieben PHP-Seiten, zwei gemeinsame Includes (Kopf- und Fußbereich), zwei JavaScript-Dateien (Buchungslogik und allgemeine Oberflächenfunktionen) sowie ein zentrales Stylesheet. Eine Datenbank wird nicht benötigt; Buchungen werden als JSON-Dateien gespeichert.

10 Verwendete KI-Systeme und Werkzeuge

Bei der Erstellung von Projekt B wurden KI-Systeme als unterstützende Werkzeuge eingesetzt. Die Offenlegung erfolgt gemäß den Projektvorgaben. Tabelle 10.1 fasst die eingesetzten KI-Systeme und ihren jeweiligen Einsatzzweck zusammen.

Tabelle 10.1: Eingesetzte KI-Systeme in Projekt B

KI-System	Anbieter	Einsatzzweck
Claude	Anthropic	Hauptsächlich genutzt zur Erstellung des Quelltexts (PHP, JavaScript, CSS) sowie der textlichen Inhalte.
Gemini	Google	Erstellung der verwendeten Bilder bzw. Grafiken (Städtebilder).

Der weit überwiegende Teil des Quelltexts und der Inhalte wurde mit Claude erstellt und anschließend vom Team geprüft, angepasst und integriert. Die auf den Seiten eingebundenen Städtebilder (images/) wurden mit Gemini generiert.

10.1 Weitere Werkzeuge

Neben den KI-Systemen kamen die folgenden Werkzeuge zum Einsatz: XAMPP als lokale Laufzeitumgebung (Apache, PHP), Git bzw. GitHub zur Versionsverwaltung sowie ein Webbrowser zum Testen der Anwendung.

11 Funktionale Umsetzung: Funktion und Skript

Analog zu Projekt A ordnet dieses Kapitel die Funktionalitäten den jeweiligen Skripten zu (Tabelle 11.1).

Tabelle 11.1: Zuordnung Funktionalität zu Skript (Projekt B)

Funktionalität	Skript / Datei	Kurzbeschreibung
Buchung		
Buchungsformular	booking.php	Eingabemaske für die Buchung
Echtzeit-Kostenberechnung	js/booking.js	Personen $\times$ Tage $\times$ Preis + Ausflüge
Dynamische Personenfelder	js/booking.js	Felder je nach Personenzahl
Clientseitige Validierung	js/booking.js	Pflichtfelder und Datumsprüfung
Buchungsverarbeitung	process_booking.php	POST auswerten, Preis berechnen, Kartennummer maskieren
Datenspeicherung (JSON + Zeitstempel)	process_booking.php	Ablage je Buchung unter bookings/
Buchungsbestätigung	confirmation.php	Zusammenfassung aus der Session
GET-/POST-Demonstration	formcheck.php	Test der Formularübertragung
Oberfläche und Navigation		
Hell-/Dunkel-Modus	js/main.js	Umschaltung, gespeichert im localStorage
Mobiles Menü / Navigation	js/main.js, includes/header.php	responsive Navigation
News-Ticker / Besucherzähler	includes/footer.php	Zählung über visitor_count.txt

Fortsetzung nächste Seite

Tabelle 11.1 – Fortsetzung

Funktionalität	Skript / Datei	Kurzbeschreibung
Gemeinsamer Kopf-/Fußbereich	includes/header.php, includes/footer.php	wiederverwendbare Bausteine
Gestaltung / Theme	css/style.css	Farbschema, Layout, Hell-/Dunkel-Modus
Inhaltsseiten		
Kundenstimmen	tours.php	mehrere Bewertungen
Teamseite	about.php	Vorstellung des fiktiven Teams
Rechtliche Seiten	legal.php	Impressum, Datenschutz, AGB

11.1 Echtzeit-Kostenberechnung

Die Buchungsseite berechnet den Gesamtpreis ohne Neuladen der Seite. Bei jeder Änderung der Eingaben (Personenzahl, Reisedauer, ausgewählte Ausflüge) wird der Preis neu ermittelt und als Zusammenfassung angezeigt. Die Grundkosten ergeben sich aus Personenzahl, Anzahl der Tage und dem Tagespreis; hinzu kommen die Kosten der gewählten Ausflüge.

```

1 var baseCost = persons * days * pricePerDay; // Grundpreis
2 var excursionCost = 0; // Summe der Ausfluege
3 // ... Aufsummieren der ausgewaehlten Ausfluege ...
4 var total = baseCost + excursionCost; // Gesamtpreis

```

Quelltext 11.1: Kern der Kostenberechnung (Auszug js/booking.js)

11.2 Speicherung der Buchungen

Bei Absenden des Formulars wertet `process_booking.php` die übermittelten Daten aus, berechnet den Gesamtpreis erneut serverseitig und maskiert die angegebene Kreditkartennummer. Jede Buchung wird anschließend mit einem Zeitstempel als eigene JSON-Datei im Verzeichnis `bookings/` gespeichert (Dateiname nach dem Muster `ST-JJJJMMTT-XXXXXX.json`). Die Zusammenfassung wird über die Session an die Bestätigungsseite übergeben.

12 Verzeichnisstruktur (Backend / Webserver)

Wie Projekt A wird auch SunShine Tours vollständig in das Webserver-Wurzelverzeichnis kopiert. Die folgende Übersicht zeigt die Struktur:

```
sunshine-tours/
|-- index.php Startseite
|-- about.php Ueber uns / Team
|-- tours.php Touren und Kundenstimmen
|-- booking.php Buchungsformular
|-- process_booking.php Verarbeitung + Speicherung der Buchung
|-- confirmation.php Buchungsbestaetigung
|-- legal.php Impressum / Datenschutz / AGB
|-- formcheck.php GET-/POST-Testseite
|-- visitor_count.txt Zaehlerstand Besucherzaehler (beschreibbar)
|
|-- includes/
| |-- header.php Kopfbereich + Navigation + Theme-Schalter
| |-- footer.php Fussbereich + News-Ticker + Besucherzaehler
|
|-- js/
| |-- booking.js Echtzeit-Kosten, dynamische Felder, Validierung
| |-- main.js Hell-/Dunkel-Modus, mobiles Menue
|
|-- css/
| |-- style.css Farbschema, Layout, responsive Regeln
|
|-- images/
| |-- paris.jpg, bielefeld.jpg, chernobyl.jpg,
| |-- gary.jpg, pyongyang.jpg (KI-generierte Bilder)
|
|-- bookings/
|   |-- ST-JJJJMMTT-XXXXXX.json gespeicherte Buchungen (beschreibbar)
```

Tabelle 12.1 fasst die Aufgaben der Komponenten zusammen.

Tabelle 12.1: Komponenten von Projekt B

Datei / Verzeichnis	Aufgabe
index.php	Startseite mit Hero-Bereich und Kennzahlen.
about.php	Vorstellung des fiktiven Teams und der Werte.

Datei / Verzeichnis	Aufgabe
tours.php	Tourangebote und Kundenbewertungen.
booking.php	Buchungsformular mit Echtzeit-Kostenanzeige.
process_booking.php	Wertet die Buchung aus, berechnet den Preis, maskiert die Kartenummer und speichert die Buchung als JSON.
confirmation.php	Zeigt die Zusammenfassung der Buchung.
legal.php	Impressum, Datenschutz und AGB.
formcheck.php	Demonstration der GET-/POST-Verarbeitung.
includes/	Gemeinsamer Kopf- und Fußbereich (Navigation, Theme-Schalter, News-Ticker, Besucherzähler).
js/	Clientlogik: Buchungsberechnung (booking.js) und Oberflächenfunktionen (main.js).
css/	Zentrales Stylesheet inkl. Hell-/Dunkel-Modus.
images/	KI-generierte Städtebilder.
bookings/	Ablageverzeichnis der gespeicherten Buchungen (zur Laufzeit beschrieben).
visitor_count.txt	Persistenter Zählerstand des Besucherzählers.

13 Installation

13.1 Systemvoraussetzungen

SunShine Tours benötigt lediglich eine Standard-XAMPP-Umgebung mit Apache und PHP. Eine Datenbank wird *nicht* benötigt, da die Daten dateibasiert gespeichert werden. Besondere PHP-Erweiterungen sind nicht erforderlich.

13.2 Installationsschritte

1. **Projekt kopieren.** Den Projektordner in das Webserver-Wurzelverzeichnis legen (bei XAMPP: C:\xampp\htdocs).
2. **Schreibrechte prüfen.** Das Verzeichnis bookings/ sowie die Datei visitor_count.txt müssen für den Webserver beschreibbar sein, damit Buchungen gespeichert und der Besucherzähler aktualisiert werden können.
3. **Aufruf.** Die Anwendung über `http://localhost/<Projektordner>/index.php` im Browser öffnen.

13.3 Zugangsdaten

Für Betrieb und Bewertung sind **keine Zugangsdaten erforderlich**. Die Anwendung besitzt weder eine Anmeldung noch eine Datenbank. Die im Buchungsformular abgefragten Zahlungsdaten dienen ausschließlich Demonstrationszwecken; die Kreditkartennummer wird vor der Speicherung maskiert. Die gespeicherten Buchungen liegen als JSON-Dateien im Verzeichnis bookings/.

14 Fazit

Mit GuardX (Projekt A) und SunShine Tours (Projekt B) wurden zwei unterschiedlich akzentuierte Web-Anwendungen umgesetzt. Projekt A demonstriert den sicheren Umgang mit Sessions, Authentifizierung, Rechteverwaltung und Datenbankzugriffen und bündelt mehrere praxisnahe Sicherheitswerkzeuge unter einer gemeinsamen Oberfläche. Projekt B zeigt den Aufbau einer responsiven, dateibasierten Web-Seite mit clientseitiger Echtzeit-Logik und legt die dabei eingesetzten KI-Werkzeuge offen.

Beide Anwendungen erfüllen die jeweils geforderten Funktionen und sind, wie in den Installationskapiteln beschrieben, auf einer Standard-XAMPP-Umgebung lauffähig. Die Zuordnung der Funktionen zu den Skripten (Kapitel 7 und 11) macht den Aufbau nachvollziehbar und erleichtert die Bewertung.

Literatur

- [1] Troy Hunt. *Have I Been Pwned: Pwned Passwords (Range-API, k-Anonymity)*. 2024. URL: <https://haveibeenpwned.com/API/v3#PwnedPasswords> (besucht am 24.06.2026).
- [2] OWASP Foundation. *Cross-Site Request Forgery Prevention Cheat Sheet*. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html (besucht am 24.06.2026).
- [3] The PHP Group. *PHP-Handbuch: password_hash*. 2024. URL: <https://www.php.net/manual/de/function.password-hash.php> (besucht am 24.06.2026).
- [4] The PHP Group. *PHP-Handbuch: Prepared Statements (MySQLi)*. 2024. URL: <https://www.php.net/manual/de/mysqli.quickstart.prepared-statements.php> (besucht am 24.06.2026).